Reasoning About Programs

Assessed PMT - Structural Induction

Answers to all questions to be submitted to Scientia by 7pm on Monday 23rd Feb

Sophia Drossopoulou and Mark Wheelhouse

Aims This exercise sheet is about structural induction on lists and trees, the correct set up of the proofs, and the correct use of quantifiers. It is also about the discovery of necessary auxiliary lemmas, and about finding the appropriate inductive principle given a data type definition.

In some questions you may find it helpful to use some of the the following list lemmas:

(A) $\forall$`xs:[a], ys:[a]. length (xs++ys) = (length xs) + (length ys)`

(B) $\forall$`x:a, xs:[a]. ys:[a]. (x:xs)++ys = x:(xs++ys)`

(C) $\forall$`x:a. [x] = x:[]`

(D) $\forall$`xs:[a], ys:[a], zs:[a]. xs++(ys++zs) = (xs++ys)++zs`

(E) $\forall$`x:a, xs:[a]. [x]++xs = x : xs`

1st Question (1 point):

Consider the following definitions, where `enc` flattens a tree into a list of `Code`s:

```
data Code = Lf Int | Nd
data Tree = Node Tree Tree | Leaf Int

enc :: Tree -> [Code]
enc (Leaf i) = [ Lf i ]
enc (Node t1 t2) = (Nd:(enc t1))++(enc t2)
```

Write out the values of the following terms:

a) `enc (Node (Leaf 3) (Node (Leaf 5) (Leaf 7)))`

b) `enc (Node (Node (Leaf 3) (Leaf 5)) (Leaf 7))`

2nd Question (10 points):

The following functions count the number of elements in lists or trees

```
length :: [a] -> Int
length [] = 0
length (z:zs) = 1 + length zs

size :: Tree -> Int
size (Leaf i) = 1
size (Node t1 t2) = 1 + (size t1) + (size t2)
```

Prove that

(i) $\forall$`t:Tree. length(enc t) = size t`

In your proof be sure to state what is given, what is to be shown and justify each step.

3rd Question (1 point):

Consider now the function `dec` which re-creates a tree out of the flattened representation:

```
dec :: [Code] -> Tree
dec cds = t
       where
       (t, []) = decAux cds

decAux :: [Code] -> (Tree, [Code])
decAux  ((Lf i):cds) = ( Leaf i,  cds )
decAux  (Nd:cds)   = ( Node t1 t2, cds2)
      where
      (t1, cds1) = decAux cds
      (t2, cds2) = decAux cds1
```

Note that `dec` is a partial funciton and is therefore not defined for *all* possible inputs.

What is the value of

a) `decAux [(Lf 3),Nd,(Lf 5),(Lf 7)]`

b) `decAux [Nd,(Lf 3),(Lf 5),(Lf 7)]`

c) `dec [(Lf 3),Nd,(Lf 5),(Lf 7)]`

d) `dec [Nd,(Lf 3),(Lf 5)]`

4th Question (12 points)

We want to prove that the functions `enc` and `dec` are inverses, i.e that

$$\text{(ii)} \quad \forall \texttt{t:Tree}.\ \texttt{dec (enc t)} = \texttt{t}$$

Obviously, we need to prove that

$$\text{(iii)} \quad \forall \texttt{t:Tree}.\ \texttt{decAux (enc t)} = \texttt{(t,[])}$$

However, (`iii`) is too weak to be directly proven by induction. Therefore, we strengthen (`iii`) to describe more general properties of the function `decAux`. Therefore, we want to prove:

$$\text{(iv)} \quad \forall \texttt{t:Tree}.\ \forall \texttt{cds:[Code]}.\ \texttt{decAux ((enc t)++cds)} = \texttt{(t,cds)}$$

which implies (`iii`).

Prove (`iv`) by structural induction. State what is to be shown and justify each step.

Remarks:

R1 You need to take some care with the definition and application of the induction hypothesis.

R2 Assertion (`iv`) clarifies how the program works: `decAux` splits its input into the part that is a an encoded tree and the remainder; it then returns the pair consisting of the tree and the remaining input.

5th Question (3 points):

Now consider a different pair of encode/decode functions:

```
encd :: Tree -> [Code]
encd (Leaf i) = [ Lf i ]
encd (Node t1 t2) =  (encd t1)++(encd t2)++ [Nd]

decd :: [Code] -> Tree
decd cds = decdAux cds []

decdAux :: [Code] -> [Tree] -> Tree
decdAux [] t:ts  =  t
decdAux ((Lf i):cds) ts  =  decdAux cds  ((Leaf i):ts)
decdAux (Nd:cds) (t1:t2:ts)  =  decdAux cds  ((Node t2 t1):ts)
```

Write out the values of the following terms:

a) `encd (Node (Leaf 3) (Node (Leaf 5) (Leaf 7)))`

b) `encd (Node (Node (Leaf 3) (Leaf 5)) (Leaf 7))`

c) `decdAux [(Lf 9)] [(Node (Leaf 5) (Leaf 7)),(Leaf 3)]`

d) `decdAux [(Lf 5),(Lf 7),Nd,(Lf 9)] [(Leaf 3)]`

6th Question (7 points) - CHALLENGE:

We want to prove that the functions `encd` and `decd` are inverses, i.e that

$$(\mathtt{v}) \quad \forall \mathtt{t:Tree}.\ \mathtt{decd\ (encd\ t)} = \mathtt{t}$$

As in the previous question, proving (`v`) requires proving another assertion, namely:

$$(\mathtt{vi}) \quad \forall \mathtt{t:Tree}.\ \mathtt{decdAux\ (encd\ t)\ []} = \mathtt{t}$$

This assertion is too weak to be directly proven by induction. Write, *but do not prove*, a stronger assertion (`vii`) which *can* be proven by induction. Outline how (`vii`) implies (`vi`).

Hint: The following thoughts may help to find (`vii`). Notice that the function `decdAux` returns only if its first argument is an empty list, otherwise it calls itself. Therefore, we can prove (`vi`) if we have (`viii`) $\forall \mathtt{t:Tree}.\ \mathtt{decdAux\ (encd\ t)\ []} = \mathtt{decdAux\ []\ t:[]}$
However, (`viii`) is too weak, and cannot be proven by induction. As we saw in the Week 4 PMT Question 2, and in the lectures, we need to generalize such assertions, so that they talk about many more input values to the function `decdAux`, i.e. generalize the empty lists which appear in (`viii`).

7th Question (6 points):

Consider the following data type `Map a b`, and assume a property $P \subseteq$ `Map a b`.

```
data Map a b = None | Last a | Comb b (Map a b)
```

Write the structural induction principle which can demonstrate the validity of $\forall \mathtt{m:Map\ a\ b}.\ P(\mathtt{m})$.

Comparison of the sheet's questions with possible exam questions

Questions 1, 3, and 5 are more straightforward, and they aim to help you think about Question 2, 4, and 6 respectively. Such questions may appear at the beginning of an exam question. Question 2 is representative of half or a third of an exam question. Question 4 is representative of a more challenging half exam question. Question 6 is an interesting challenge question, a question like it might appear as a very advanced part of an exam paper. Question 7 could appear as part of an exam paper.

Thank you

to Tristan Allwood and Alexander J. Summers for feedback and advice on the development of these questions.